

4 Sorting

Sorting is one of the most fundamental topics in Data Structures and Algorithms (DSA). It is usually taught immediately after topics like:

- Time Complexity
- Arrays

This is because sorting helps build algorithmic thinking and introduces many important problem-solving techniques.

Why is Sorting Important?

1. Sorting Algorithms Teach Important Techniques

There are many sorting algorithms, such as:

- Bubble Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Counting Sort

Each algorithm teaches an important concept or algorithmic paradigm.

Examples

Merge Sort

Teaches:

- **Divide and Conquer**

2. Many Algorithms Require Sorted Input

A large number of algorithms only work when data is sorted.

Example: Binary Search

Binary Search works only if:

- The array is already sorted

Without sorting, binary search cannot function correctly.

3. Sorting Helps Discover Patterns in Problems

Sometimes while solving problems:

- You may get completely stuck
- No idea comes to mind
- Existing algorithms do not help immediately

Why Does This Help?

Sorting organizes data in a structured manner.

This helps in:

- Detecting duplicates
- Finding pairs/triplets
- Using two-pointer techniques
- Applying binary search
- Observing mathematical patterns

What is Sorting?

Sorting means:

Arranging elements of an array in a particular order.

The order can be:

- **Ascending Order** → Smallest to Largest
- **Descending Order** → Largest to Smallest

Example of Sorting

Unsorted Array

[5,1,4,3,2]

After Sorting in Ascending Order

[1,2,3,4,5]

Sorting Can Be Applied On

1. Numbers

Numbers can be compared directly.

Example:

- Integer arrays
- Floating-point arrays

2. Strings

Strings can be sorted:

- Lexicographically (dictionary order)
- By length
- By custom rules

3. Custom Objects

Custom objects can also be sorted.

Example:

Student Objects

Can be sorted based on:

- Marks
- Roll Number
- Name
- Attendance

Bubble Sort

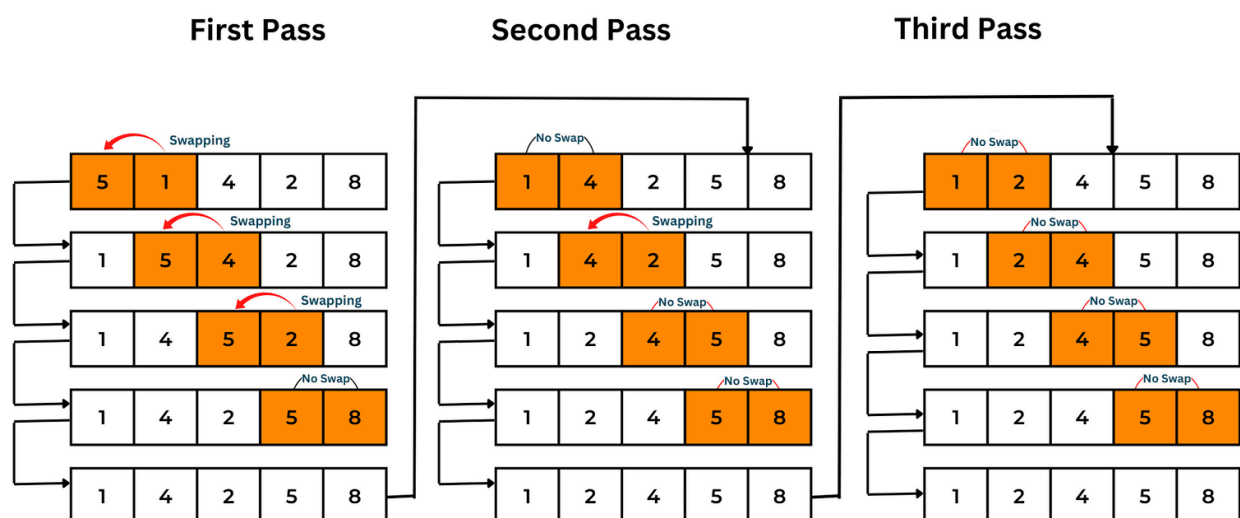
Bubble Sort is one of the most basic and beginner-friendly sorting algorithms.

It works by:

Comparing adjacent (consecutive) elements

and swapping them if they are in the wrong order.

BUBBLE SORTING



```
array = [30,20,60,40,10,50]
n = len(array)
print(array,"before bubble sort")
```

```

for i in range(n):
    for j in range(n-i-1):
        if array[j] > array[j+1]:
            array[j],array[j+1] = array[j+1],array[j]
print(array,"after bubble sort")

```

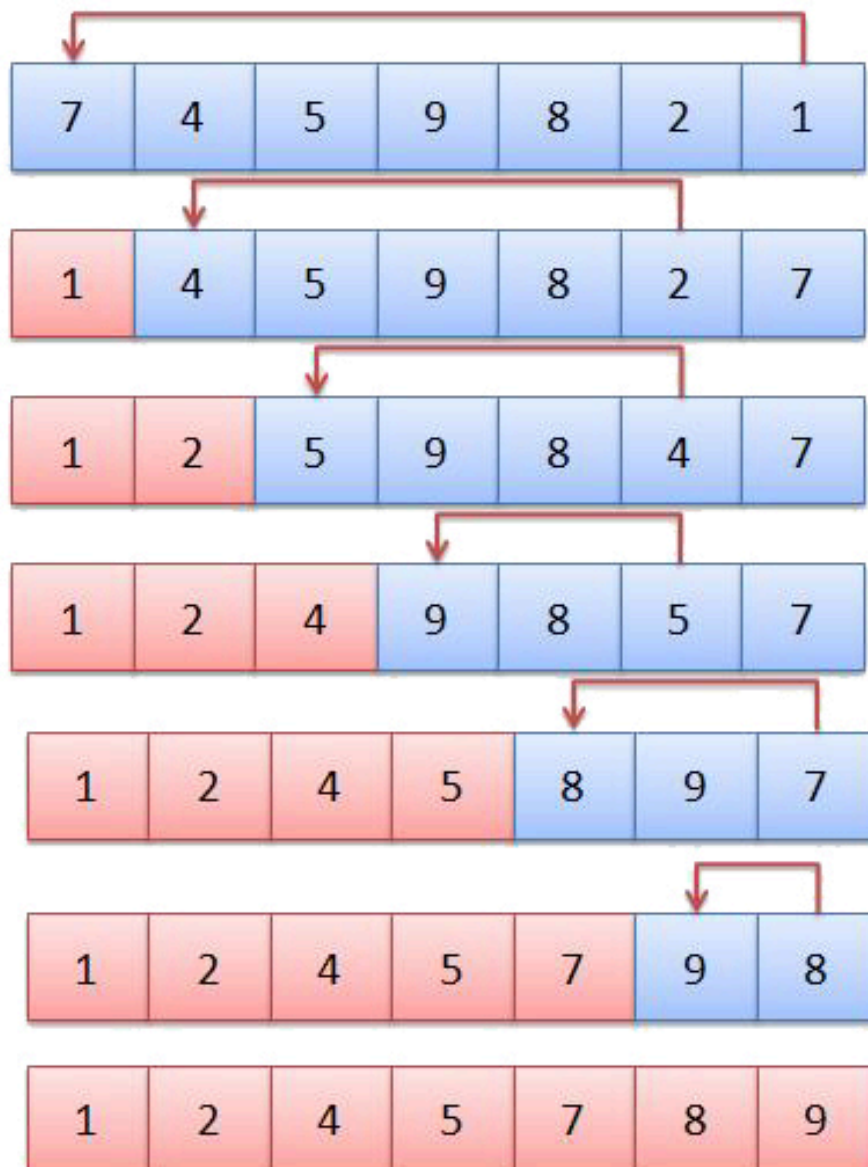
Selection Sort

Selection Sort is one of the:

- Simplest
- Most beginner-friendly sorting algorithms.

It is even simpler than Bubble Sort in terms of understanding.

Repeatedly select the smallest element from the unsorted part and place it at the beginning.



```

array = [30,20,60,40,10,50]
n = len(array)
print(array,"before selection sort")

```

```

for i in range(n):
    smallestEle = i
    for j in range(i+1,n):
        if array[j] < array[smallestEle]:
            smallestEle = j
    array[i],array[smallestEle] = array[smallestEle],array[i]
print(array,"after selection sort")

```

Insertion sort

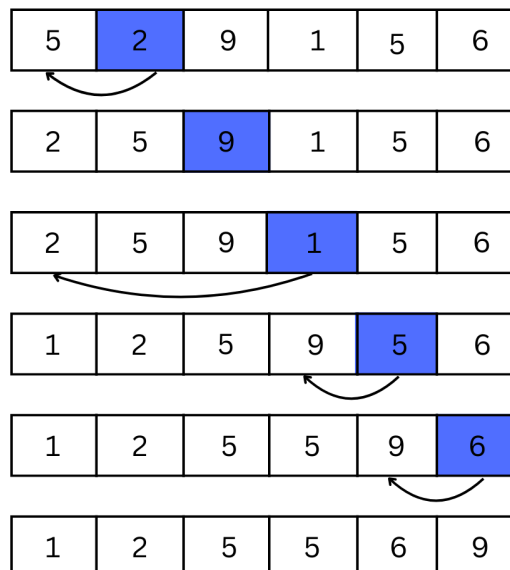
Insertion Sort divides the array into two parts:

- **Sorted Part**
 - **Unsorted Part**
- Initially:
- The **first element** is always considered sorted.
 - Remaining elements are unsorted.

Working Principle

Insertion Sort works by:

1. Picking an element from the unsorted part.
2. Placing it into its correct position in the sorted part.
3. Shifting larger elements one position to the right.



```

array = [30,20,60,40,10,50]
n = len(array)
print(array,"before insertion sort")
for i in range(1,n):
    j = i-1

```

```

key = array[i]
while (j >= 0 and array[j] > key):
    array[j+1] = array[j]
    j = j-1
array[j+1] = key
print(array, "after insertion sort")

```

What is Merge Operation?

Merge operation is a technique used to combine **two sorted arrays** into **one sorted array**. It is the most important concept used in **Merge Sort**.

Input and Output

We are given:

- Array A → Sorted
- Array B → Sorted
- We need to create:
- Array C → Also Sorted

```

a = [2,5,7,9,11]
b = [3,4,8,10]
c = [2, 3, 4, 5, 7, 8, 9, 10, 11]

```

```

a = [2,5,7,9,11]
b = [3,4,8,10]
n = len(a) + len(b)
c = [0]*n
i = 0
j = 0
k = 0
while(i < len(a) or j < len(b)):
    if i < len(a) and j < len(b):
        if a[i] <= b[j]:
            c[k] = a[i]
            i = i+1
        else:
            c[k] = b[j]
            j = j+1
    elif i < len(a):
        c[k] = a[i]
        i = i+1
    else:
        c[k] = b[j]
        j = j+1

```

```
k = k + 1  
print(c)
```

What is Merge Sort?

Merge Sort is a:

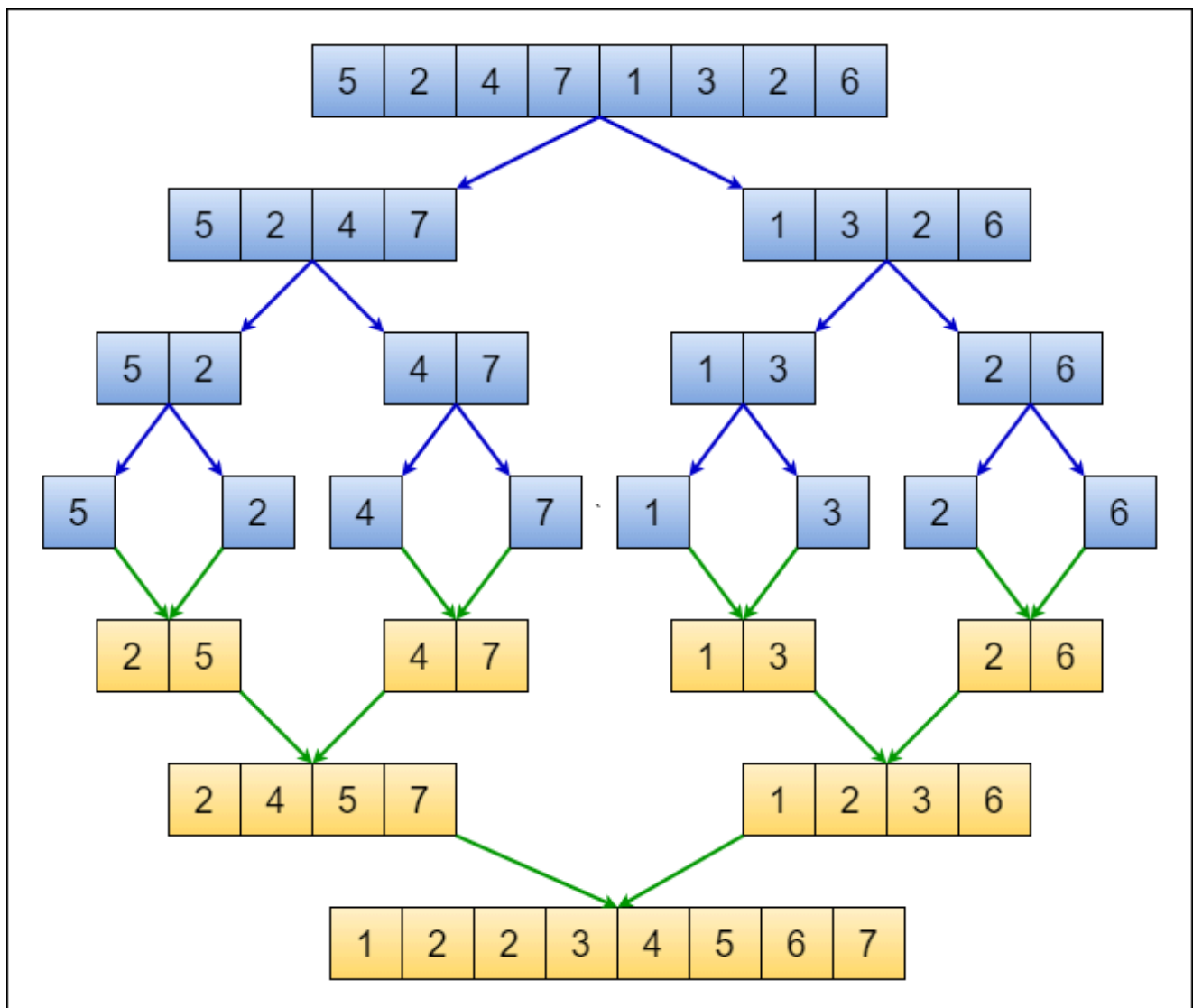
Divide and Conquer Sorting Algorithm

It divides the array into smaller parts, sorts them, and merges them back.

Core Idea of Merge Sort

Merge Sort works in 3 major steps:

Step	Work
Divide	Split array into smaller parts
Conquer	Sort smaller parts recursively
Merge	Combine sorted parts



```

a = [4,6,3,8,1,9,2,5,7]
def merge(start,mid,end):
    i = start
    j = mid + 1
    k = 0
    c = [0]*(end-start+1)
    while i<=mid or j<=end:
        if i<=mid and j<=end:
            if a[i] <= a[j]:
                c[k] = a[i]
                i = i+1
            else:
                c[k] = a[j]
                j = j+1
        elif (i<=mid):
            c[k] = a[i]
            i = i+1
        else:
            c[k] = a[j]
            j = j+1
        k = k+1
    for i in range(len(c)):
        a[start + i] = c[i]

def mergeSort(start,end):
    if start < end:
        mid = (start + end) // 2
        mergeSort(start, mid)
        mergeSort(mid+1,end)
        merge(start,mid,end)

mergeSort(0,len(a)-1)
print(a)

```

What is Partitioning?

Partitioning means:

Rearranging an array around a selected element called the **key** or **pivot**.

After partitioning:

- Elements **smaller than pivot** go to the left side
- Elements **equal to pivot** stay in the middle
- Elements **greater than pivot** go to the right side

Partitioning ≠ Sorting

After partitioning:


```
[2,1,0,3,5,4,6]
```

the array is still **unsorted**.

Notice:

- 2,1,0 are not sorted
- 5,4,6 are not sorted

Partitioning only guarantees:

```
Left side < Pivot < Right side
```

It does NOT guarantee full sorting.

Pivot / Key

The selected element used for partitioning is called:

- **Key**
- **Pivot**

Both mean the same thing.

In this algorithm:

Pivot is assumed to be the LAST element of the array.

Algorithm

We maintain one variable:

```
left = -1  
No elements smaller than pivot are found yet
```

The algorithm scans array from left → right.

Whenever an element smaller than pivot is found:

1. Increase left
2. Swap current element with arr[left]

This keeps all smaller elements grouped on the left side.

Partitioning the array $a = [1, 5, 2, 6, 0, 4, 7, 3]$ with pivot = 3 (last element)

Step	i	Element (a[i])	Array	left	Action / Explanation
Initial	-	-	1 5 2 6 0 4 7 3 0 1 2 3 4 5 6 7 pivot	-1	pivot = 3 (last element) left = -1
Step 1	0	1	1 5 2 6 0 4 7 3 ↑ left = 0	0	$1 < 3 \rightarrow \text{True}$ left++ $\rightarrow 0$ swap(a[0], a[0]) $\rightarrow$ no change
Step 2	1	5	1 5 2 6 0 4 7 3	0	$5 < 3 \rightarrow \text{False}$ Do nothing
Step 3	2	2	1 2 5 6 0 4 7 3 ↑ left = 1	1	$2 < 3 \rightarrow \text{True}$ left++ $\rightarrow 1$ swap(a[2], a[1]) $\rightarrow$ swap 2 and 5
Step 4	3	6	1 2 5 6 0 4 7 3	1	$6 < 3 \rightarrow \text{False}$ Do nothing
Step 5	4	0	1 2 5 6 0 4 7 3 ↑ left = 2	2	$0 < 3 \rightarrow \text{True}$ left++ $\rightarrow 2$ swap(a[4], a[2]) $\rightarrow$ swap 0 and 5
Step 6	5	4	1 2 0 6 5 4 7 3	2	$4 < 3 \rightarrow \text{False}$ Do nothing
Step 7	6	7	1 2 0 6 5 4 7 3	2	$7 < 3 \rightarrow \text{False}$ Do nothing
Final Step Place Pivot	-	-	1 2 0 3 5 4 7 6 ↑ left = 3	3	left++ $\rightarrow 3$ swap(a[left], a[7]) $\rightarrow$ swap 3 and 6 Pivot is at index 3 (correct position)

Elements < 3	Pivot = 3	Elements > 3	
1 2 0	3	5 4 7 6	< pivot = pivot > pivot

```

a = [1,5,2,6,0,4,3]
n = len(a)
key = a[n-1]
left = -1
for i in range(n-1):
    if(a[i] <= key):
        left = left + 1
        a[i],a[left] = a[left],a[i]
left = left + 1
a[n-1],a[left] = a[left],a[n-1]
print(a)

```

What is Quicksort ?

Quicksort is a **Divide and Conquer Sorting Algorithm** that uses the **Partition Function** to sort the array.

Main Idea Behind Quicksort

The idea is very simple:

1. Choose a Pivot
2. Partition the array around the pivot
3. Pivot reaches its correct position
4. Recursively sort left part
5. Recursively sort right part

```

a = [1,5,2,6,0,4,7,3]
def partation(start,end):
    left = start - 1
    pivot = a[end]
    for i in range(start,end):
        if a[i] <= pivot:
            left = left + 1
            a[left],a[i] = a[i],a[left]
    left = left + 1
    a[end],a[left] = a[left],a[end]
    return left

def quickSort(left,right):
    if left < right:
        partationPoint = partation(left,right)
        quickSort(left,partationPoint-1)
        quickSort(partationPoint+1,right)

quickSort(0,len(a)-1)
print(a)

```

Problem 1

Given an integer array `nums` , move all 0 's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

Input: `nums = [0,1,0,3,12]`

Output: `[1,3,12,0,0]`

```

from typing import List
class Solution:
    def moveZeros(self,nums:List[int]) ->None:
        left = 0
        for i in range(len(nums)):
            if nums[i] != 0:
                nums[left],nums[i]=nums[i],nums[left]
                left = left + 1

s = Solution()
a = [0,1,0,3,12]
s.moveZeros(a)

print(a)

```

Problem 2

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`

Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: 2

To achieve this we use:

Moore's Voting Algorithm

Also called:

Boyer–Moore Voting Algorithm

Moore's Voting Algorithm (Boyer–Moore Voting Algorithm)

Find the **majority element** in an array. A majority element is an element that appears **more than $\lfloor n/2 \rfloor$** times.

PROBLEM STATEMENT

Given an array of size `n`, find the majority element. It is **guaranteed** that the majority element always exists.

Examples:

- `[2, 2, 1, 1, 1, 2, 2]` → 2 (occurs $4 > 7/2$ times)
- `[3, 2, 3]` → 3 (occurs $2 > 3/2$ times)

INTUITION

Think of each element as a vote.

- Same as current candidate → **vote for it (count++)**
- Different from candidate → **vote against it (count--)**

When count becomes 0, it means the votes are balanced, so we choose a new candidate.

Since majority element appears more than $n/2$ times, it can never be completely canceled out.

HOW IT WORKS?

1. Take the first element as candidate and set count = 1.
2. Traverse the array from the second element.
3. If current element == candidate → count++
4. Else → count--
5. If count becomes 0 → choose current element as new candidate, and set count = 1.
6. After the traversal, the candidate is the majority element.

PYTHON IMPLEMENTATION

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        candidate = nums[0]
        count = 1

        for i in range(1, len(nums)):
            if nums[i] == candidate:
                count += 1
            else:
                count -= 1
                if count == 0:
                    candidate = nums[i]
                    count = 1

        return candidate
```

Note:
The problem guarantees that majority element always exists.

STEP-BY-STEP EXAMPLE

Array: `[1, 2, 3, 2, 2, 4, 2]` (Majority Element = 2)

Index	Element (Current)	Candidate	Count	Explanation
0	1	1	1	Initialize: candidate = 1, count = 1
1	2	2	1	$2 \neq 1 \rightarrow \text{count-- (0)}$ $\text{count} == 0 \rightarrow \text{candidate} = 2, \text{count} = 1$
2	3	3	1	$3 \neq 2 \rightarrow \text{count-- (0)}$ $\text{count} == 0 \rightarrow \text{candidate} = 3, \text{count} = 1$
3	2	2	1	$2 \neq 3 \rightarrow \text{count-- (0)}$ $\text{count} == 0 \rightarrow \text{candidate} = 2, \text{count} = 1$
4	2	2	2	$2 == 2 \rightarrow \text{count++ (2)}$
5	4	2	1	$4 \neq 2 \rightarrow \text{count-- (1)}$
6	2	2	2	$2 == 2 \rightarrow \text{count++ (2)}$

Final Candidate = 2 (This is the majority element)

VISUALIZATION OF CANCELLATION IDEA

`[1, 2, 3, 2, 2, 4, 2]`

1 → 2 3 × 2 4 × 2

Remaining: 2 → Majority Element

All different elements cancel out with majority element until one majority element remains.

COMPLEXITY

- Time Complexity : $O(n)$
- Space Complexity : $O(1)$

IMPORTANT POINTS

- Works only if majority element exists.
- If no majority element exists, the result may be incorrect.
- Extremely useful for solving in $O(n)$ time and $O(1)$ space.

```
from typing import List
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        moc = nums[0]
        co = 1
        for i in range(1, len(nums)):
            if nums[i] == moc:
                co = co + 1
            else:
                co = co - 1
                if co == 0:
                    moc = nums[i]
        return moc
```

```
co = 1
```

```
return moc
s = Solution()
a = [1,2,3,2,2,4,2]
x = s.majorityElement(a)
print(x)
```

Problem 3

Given an array `nums` with `n` objects colored red, white, or blue, sort them **in-place** so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers `0`, `1`, and `2` to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

Input: `nums = [2,0,2,1,1,0]`

Output: `[0,0,1,1,2,2]`

Sort Colors (Dutch National Flag) – Dry Run Example

Initial Array: [2, 1, 0, 1, 2, 0]

n = 6

Step	i	left	right	nums[i]	Action	Array After Action	Explanation												
Initial	0	0	5	2	Start	<table><tr><td>2</td><td>1</td><td>0</td><td>1</td><td>2</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	2	1	0	1	2	0	0	1	2	3	4	5	left = 0, right = 5, i = 0
2	1	0	1	2	0														
0	1	2	3	4	5														
1	0	0	5	2	nums[i] == 2 swap nums[i] and nums[right] right--	<table><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>2</td><td>2</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	0	1	0	1	2	2	0	1	2	3	4	5	Swap 2 and 0, move right backward
0	1	0	1	2	2														
0	1	2	3	4	5														
2	0	0	4	0	nums[i] == 0 swap nums[i] and nums[left] left++, i++	<table><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>2</td><td>2</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	0	1	0	1	2	2	0	1	2	3	4	5	Swap 0 and 0 (no change), move left and i forward
0	1	0	1	2	2														
0	1	2	3	4	5														
3	1	1	4	1	nums[i] == 1 i++	<table><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>2</td><td>2</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	0	1	0	1	2	2	0	1	2	3	4	5	1 is in correct position, just move i forward
0	1	0	1	2	2														
0	1	2	3	4	5														
4	2	1	4	0	nums[i] == 0 swap nums[i] and nums[left] left++, i++	<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>2</td><td>2</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	0	0	1	1	2	2	0	1	2	3	4	5	Swap 0 and 1, move left and i forward
0	0	1	1	2	2														
0	1	2	3	4	5														
5	3	2	4	1	nums[i] == 1 i++	<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>2</td><td>2</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	0	0	1	1	2	2	0	1	2	3	4	5	1 is in correct position, just move i forward
0	0	1	1	2	2														
0	1	2	3	4	5														
6	4	2	4	2	nums[i] == 2 swap nums[i] and nums[right] right--	<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>2</td><td>2</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	0	0	1	1	2	2	0	1	2	3	4	5	Swap 2 and 2 (no change), move right backward
0	0	1	1	2	2														
0	1	2	3	4	5														
End	4	2	3	-	i > right → stop	<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>2</td><td>2</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr></table>	0	0	1	1	2	2	0	1	2	3	4	5	Array is sorted!
0	0	1	1	2	2														
0	1	2	3	4	5														

```
from typing import List
class Solution:
    def setColors(self, nums: List[int]) -> None:
        left = 0
        right = len(nums) - 1
        i = 0
        while i <= right:
            if nums[i] == 0:
                nums[i], nums[left] = nums[left], nums[i]
                left = left + 1
                i = i + 1
            elif nums[i] == 2:
                nums[i], nums[right] = nums[right], nums[i]
                right = right - 1
```

```
        right = right - 1
    else:
        i = i + 1

s = Solution()
a = [2,0,2,1,1,0]
s.setColors(a)
print(a)
```